

# Shortest Time to Completion First (STCF)

STCF runs the process with the least CPU work left. A strictly shorter ready process can interrupt the running process.

## 1. Model

The visualizer uses one CPU, whole ticks, and CPU-only processes. **Remaining service** starts at the input service time and decreases by one for every tick the process runs. Switching processes takes no time. STCF has no quantum.

Only arrived processes can run. Ready processes wait for the CPU; future processes have not arrived. The queue displays insertion order, but selection searches the whole queue.

Queue entries read from left to right; [] means empty. Each timeline cell is one tick, and a dash (-) means the CPU is idle. Examples under different rules are separate scenarios.

## 2. Scheduling rules

### Rule 1. Select the shortest remaining service

When the CPU is free, compare all ready processes and select the least remaining service. Use work left now, not original service time.

**Example.** A originally needed ten ticks and received eight before being preempted. At the next dispatch:

| Process | Original service | Already run | Remaining   |
|---------|------------------|-------------|-------------|
| A       | 10               | 8           | 2 <- select |
| B       | 3                | 0           | 3           |

### Rule 2. Preempt only for a strictly shorter process

After charging the tick that just ran, compare the current process with the ready processes. Preempt only if a ready process has strictly less remaining service. Equal remaining times keep the current process on the CPU.

**Example.** A starts at 0 with service 5. B arrives at 2, when A has three ticks left. These are separate cases:

|                  |             |                 |
|------------------|-------------|-----------------|
| B needs 2 ticks: | B: 2 < A: 3 | -> B preempts A |
| B needs 3 ticks: | B: 3 = A: 3 | -> A continues  |
| B needs 4 ticks: | B: 4 > A: 3 | -> A continues  |

**Rule 3. Preserve work when a process is preempted**

Append the interrupted process to the ready-queue tail, preserving remaining service. When selected again, it resumes from that amount. Its queue position does not override the shortest-remaining comparison.

**Example.** A and C arrive at 0 with service 6 and 8. B arrives at 2 with service 1.

At  $t=2$ , before admitting B:

CPU: A (4 left)                  Ready: [C: 8]

Admit B, then preempt A:

CPU: free                      Ready: [C: 8] [B: 1] [A: 4]

Dispatch B:

CPU: B (1 left)                  Ready: [C: 8] [A: 4]

At  $t=3$ , B finishes:

CPU: A (4 left)                  Ready: [C: 8]

**Rule 4. Break dispatch ties by arrival, then input order**

When the CPU is free, equal remaining times are resolved by earlier arrival, then input order. These tie-breakers do not displace an equally short process already running.

**Example.** The CPU is free in each of these separate cases.

Same remaining service, different arrivals:

B: 3 left, arrived at 1    <- select B

C: 3 left, arrived at 2

Same remaining service and same arrival:

Input order: [Z] [A]        <- select Z, not alphabetical A

**Rule 5. Complete at zero; idle only if nothing is ready**

Remove a process after its final CPU tick. Admit same-time arrivals before selecting another process. Idle only if no process is ready; stop when all processes finish.

**Example.** A arrives at 0 with service 2, B at 4 with service 1, and C at 5 with service 1.

Time 0    1    2    3    4    5    6

+---+---+---+---+---+---+

CPU | A | A | - | - | B | C |

+---+---+---+---+---+---+

$t=2$ : A finishes; no ready process, so the CPU idles.

$t=5$ : B finishes and C arrives; C runs immediately.

### 3. Events at a tick boundary

After accounting for the preceding tick:

1. Remove the running process if it has finished.
2. Admit all arrivals at the current time, in input order.
3. If a process is still running, check for a strictly shorter ready process. If one exists, append the interrupted process to the ready queue.
4. If the CPU is free, select the shortest remaining service, using Rule 4 for ties.

### 4. Worked example

Input order is A, B, C. All times are in ticks.

| <i>Process</i> | <i>Arrival</i> | <i>Service</i> |
|----------------|----------------|----------------|
| A              | 0              | 5              |
| B              | 1              | 2              |
| C              | 1              | 1              |

```

Time 0   1   2   3   4   5   6   7   8
      +---+---+---+---+---+---+---+---+
CPU   | A | C | B | B | A | A | A | A |
      +---+---+---+---+---+---+---+---+

```

```

t=0: A starts with 5 ticks of service.
t=1: A has 4 left; B arrives with 2, C with 1.
      C preempts A because 1 is the smallest remainder.
t=2: C finishes; B (2 left) runs before A (4 left).
t=4: B finishes; A resumes with 4 left.
t=8: A finishes; all processes are done.

```